

MedVet

Sistema Integral de Gestión Veterinaria

Informe técnico de proyecto: alcance, arquitectura, estado actual y hoja de ruta

14

SERVICIOS API

11

TABLAS BD

38

COMPONENTES /
PÁGINAS VUE

59

COMMITTS (5
SEMANAS)

Fecha: 16 de agosto de 2026 · **Repositorio:** medvet · **Versión:** 1.0-dev

Contenido

- 1. Visión del producto — qué es y para quién
- 2. Módulos funcionales — reservas, EHR, carnet, guardia, administración
- 3. Stack tecnológico — frontend, backend, base de datos, infraestructura
- 4. Arquitectura del sistema — capas, servicios y flujo de datos
- 5. Base de datos — esquema, dominios y relaciones
- 6. Métricas del proyecto — código, actividad y documentación
- 7. Estado actual — fortalezas y deuda técnica
- 8. Hoja de ruta — módulos planificados (Fase 1) y prioridades
- 9. Suite de documentación formal — PRD, TRD, diseño, flujos, plan de implementación
- 10. Anexos — A: Casos de uso · B: Relación de módulos · C: Modelo Entidad-Relación

1. Visión del producto

MedVet es una plataforma web para clínicas veterinarias, hospitales 24 horas y centros de especialidades. Cubre el ciclo completo de atención: desde la autogestión de citas por parte del tutor hasta la historia clínica electrónica, el visor médico de estudios por imagen, el carnet sanitario con QR y el relevo de guardia del personal.

El sistema define cuatro roles operativos: `client` (tutor), `receptionist`, `veterinarian` y `admin`, con autenticación JWT (validéz de 7 días) sobre estrategias local y token.

Público	Superficie	Acceso
Tutor / dueño de mascota	Portal público: registro, mascotas, reserva, historial, pagos	Web responsive, sesión propia
Recepción	Panel admin: citas, caja, actas de guardia	Panel /admin, rol receptionist
Veterinario / guardia	Ficha clínica, EHR, cirugía, vacunación, guardia 24/7	Panel /admin, rol veterinarian
Administración	Usuarios, servicios, horarios, reportería	Panel /admin, rol admin
Público general	Carnet sanitario verificable vía QR	/carnet/:id sin autenticación

2. Módulos funcionales

2.1 Reservas y agenda

Flujo de reserva en 4 pasos (mascota → servicio → profesional → fecha/hora) con validación de disponibilidad en tiempo real. El servicio `available-slots` calcula los turnos libres cruzando `schedules` (disponibilidad semanal por profesional) contra `appointments` existentes; previene solapamiento y respeta la duración de cada servicio. Estados de cita: `pending` · `confirmed` · `completed` · `cancelled`.

2.2 Historia clínica electrónica (EHR)

Registro de consultas con anamnesis, constantes fisiológicas (peso, temperatura, FC, FR, mucosas, TLLC), diagnóstico presuntivo/definitivo, plan terapéutico y prescripción. Incluye cirugías (valoración preoperatoria, protocolo anestésico, técnica, órdenes post-operatorias) y adjuntos clínicos categorizados (radiografía, ecografía, sangre, orina, biopsia). El frontend incorpora un visor de estudios interactivos.

2.3 Vacunación y carnet sanitario con QR

Esquemas de vacunación y desparasitación con lote, laboratorio, fecha de aplicación y revacunación calculada. El servicio `public-carnet` expone un endpoint público (sin login) que retorna mascota, tutor, vacunas, últimas historias y cirugías, junto con los datos de la clínica y un sello de verificación temporal.

2.4 Guardia 24/7 y relevo de turno

Actas de entrega de guardia (`shift-handovers`): veterinario saliente/entrante, turnos, contadores de ingresos, cirugías, urgencias y altas, notas de pacientes críticos y tareas pendientes.

2.5 Panel administrativo

Gestión de usuarios y cuerpo médico, catálogo de servicios con tarifas y duraciones, horarios por profesional, gestión de citas, facturación y reportería.

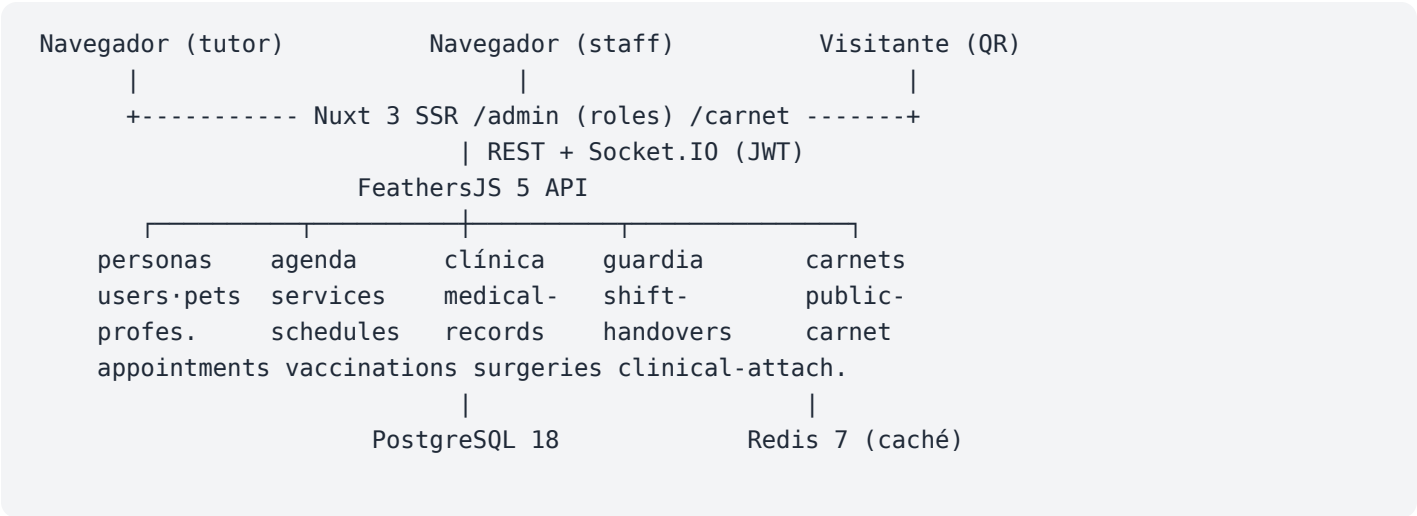
Módulo	Estado del backend	Estado del frontend
Reservas y agenda	funcional	funcional
EHR, cirugías, vacunación, adjuntos	funcional	funcional
Carnet QR público	funcional	funcional
Guardia 24/7	funcional	funcional
Facturación y caja	inexistente	maqueta con datos ficticios
Reportería y auditoría	inexistente	maqueta con datos ficticios

3. Stack tecnológico

Capa	Tecnología	Notas
Frontend	Nuxt 3.12 (Vue 3, SSR)	Modo híbrido, SEO, hot-reload
	Nuxt UI 3	UButton, UCard, UModal, UTable, UForm + zod
	Pinia	Estado global (autenticación, reserva)
	Feathers client (REST + Socket.IO)	rest-client, socketio-client, authentication-client
	Utilidades	qrcode (carnet), file-saver, docx (exportación Word)
	CSS propio + design tokens	main.css (555 líneas) con variables, modo oscuro completo
Backend	FeathersJS 5 + TypeScript	express, socketio, authentication, authentication-local
	Knex 3 + PostgreSQL 18	Migraciones, seeds, paginación
	Redis 7 (ioredis 5)	Caché y pub/sub en tiempo real
	Seguridad	bcryptjs (hash de contraseñas), helmet, cors, compression
	Logging	Winston
Infraestructura	Docker Compose	postgres, redis, pgadmin, directus (herramienta de datos)
	Render (render.yaml)	Despliegue backend desde Dockerfile
	Railway (railway.json)	Despliegue alternativo configurado
	dev.sh	Arranque de entorno local completo

4. Arquitectura del sistema

Arquitectura cliente-servidor en tres capas. El frontend consume la API por REST (fetch) y mantiene canales Socket.IO para eventos en tiempo real (actualización de citas). Los servicios Feathers usan el adaptador `@feathersjs/knex` sobre PostgreSQL, con paginación global y hooks de registro de errores.



Los 14 servicios de la API: `users` , `pets` , `services` , `professionals` , `schedules` , `appointments` , `available-slots` , `medical-records` , `clinical-attachments` , `vaccinations` , `surgeries` , `shift-handovers` , `public-carnet` y `clinical-upload` .

5. Base de datos

PostgreSQL 18 con UUIDs (`gen_random_uuid()`). Dos migraciones agrupan 11 tablas en 4 dominios + guardia. Índices en claves foráneas y columnas de consulta frecuente (fechas, estados).

Dominio	Tablas	Relaciones principales
Personas	users, pets, professionals	users 1:N pets · users 1:1 professionals
Agenda	services, schedules, appointments	appointments N:1 con users, pets, professionals, services · schedules N:1 professionals
Clínica (EHR)	medical_records, clinical_attachments, vaccinations, surgeries	Todas N:1 con pets; medical_records enlaza profesional y cita; attachments y surgeries enlazan con historias
Guardia	shift_handovers	N:1 con professionals (saliente y entrante)

6. Métricas del proyecto

59

COMMITTS

~30.100

LÍNEAS FRONTEND
(VUE/TS)

~1.600

LÍNEAS BACKEND
(TS)

5 sem.

DESARROLLO ACTIVO

Indicador	Detalle
Período de desarrollo	10-jul-2026 → 15-ago-2026 (actividad continua)
Frontend	38 archivos .vue: 9 páginas públicas, 10 páginas admin/dashboard, 14 componentes, 2 layouts
Backend	14 servicios, 2 migraciones (11 tablas), 2 seeds, autenticación JWT
Documentación	10 documentos: PRD, TRD, brief UI/UX, flujos de app, arquitectura de esquema, plan de implementación, módulos de facturación/reportes, EHR, esquema de BD, guía de despliegue Railway
Diagramas	3 diagramas Excalidraw: casos de uso, relación de módulos, entidad-relación (docs/diagrams/)
Tests automatizados	Vitest configurado; sin suites escritas todavía

7. Estado actual: fortalezas y deuda técnica

Fortalezas

- **Núcleo clínico real y conectado:** EHR, cirugías, vacunas y adjuntos persisten en PostgreSQL con servicios completos y migraciones versionadas.
- **Frontend pulido:** diseño propio con design tokens, modo oscuro auditado (16 páginas pasan contraste WCAG), widget de tasa BCV, visores médicos.
- **Documentación de nivel profesional:** PRD, TRD y diagramas que permiten incorporar nuevos desarrolladores o presentar a clientes.
- **Despliegue listo:** Docker Compose local + configuraciones Render/Railway para producción.
- **Carnet público con QR:** diferenciador comercial, endpoint sin autenticación con sello de verificación.

Deuda técnica (ordenada por impacto)

#	Deuda	Impacto	Riesgo
1	Facturación y reportería sin backend (datos ficticios en el frontend, ej. reports/index.vue:528)	La clínica no puede cobrar ni auditar ingresos reales	crítico para venta
2	Cero tests automatizados	Regresiones silenciosas en reservas y clínica	alto

3	Sin multi-tenancy (datos de la clínica hardcoded en <code>public-carnet</code>)	Un solo cliente por instalación; no escala como SaaS	medio-alto
4	Responsive móvil en progreso (hero y controles del header desbordan en 375px)	UX degradada en celulares, 60%+ del tráfico	medio
5	CSS artesanal extenso sin sistema de componentes	Mantenimiento costoso, estilos duplicados	medio-bajo
6	Adjuntos clínicos sin storage de objetos dedicado	Crecimiento de la base de datos con archivos pesados	medio-bajo

8. Hoja de ruta — Fase 1 (próximos módulos)

Módulo	Qué agrega	Integración con lo existente
Facturación y reportes reales	Facturas persistidas, IVA, métodos de pago, exportación PDF/Word/Excel/CSV	Base de todo lo demás; reemplaza las maquetas actuales
Inventario / Farmacia	Stock de medicamentos, alertas de vencimiento, movimientos de stock	Enlaza con factura: venta de producto descuenta stock
Laboratorio interno	Hemogramas, bioquímica, resultados estructurados	Extiende <code>clinical_attachments</code> con valores por estudio
Pagos online	Pasarela de pagos (MercadoPago/Stripe), webhooks	Depende de facturas reales (módulo 1)
Recordatorios de citas	Email/SMS/WhatsApp automáticos 48 h antes	Reduce ausentismo; usa <code>appointments</code> existentes

Prioridad recomendada: facturación real primero (desbloquea la venta del producto), luego tests e inventario/laboratorio, y pagos + recordatorios sobre esa base.

Suite de documentación formal

El repositorio incluye diez documentos técnicos (carpeta `docs/`) que describen el producto desde requerimientos hasta despliegue. A continuación un resumen por documento y, al final, una verificación de coherencia entre lo documentado y lo implementado en el código.

Documento	Contenido
1-PRD.md	Visión de producto, 4 perfiles de usuario (tutor, veterinario, recepcionista, admin), 12 requerimientos funcionales (auth/RBAC, mascotas, slots sin solapamiento, historia clínica, carnet, pagos multimonedas, reportaría), requerimientos no funcionales (latencia < 150 ms, LCP < 1.8 s, WCAG 2.1 AA) y 4 reglas de negocio (duración/solapamiento, cancelación hasta 2 h antes, cálculo de revacunación, facturación USD→Bs con Cashea 40 % inicial).
2-TRD.md	Arquitectura cliente-servidor reactiva, stack detallado Front/Back, tabla de endpoints con roles permitidos, algoritmo de <code>available-slots</code> en 8 pasos, canales WebSocket (<code>authenticated</code> , <code>admin-clinic</code>), pipeline de hooks de seguridad (JWT HS256, <code>protect('password')</code> , restricción de recursos propios) y variables de entorno.
3-UI-UX-Design-Brief.md	Sistema de diseño «Nuxt Emerald & Deep Space» con filosofía de marca, paleta de tokens CSS (forest, joy, semánticos), modo oscuro y modo claro, y contenedor panorámico de 1440 px.
4-App-Flow.md	Cinco user journeys con diagramas Mermaid: registro/alta de mascota, agendamiento con caché Redis, atención clínica y carnet, facturación multimonedas con Cashea, y reportaría con exportación.
5-Backend-Schema-Architecture.md	Diagrama ER completo de 11 tablas, catálogo de servicios Feathers con autenticación y roles por endpoint (v2.0 EHR & Guardia).
6-Implementation-Plan.md	Seis fases entregadas (arquitectura+auth, motor de agenda, ficha/carnet, facturación multimonedas, layout widescreen, documentación) + Fase 7 en planificación (tests > 85 %, Playwright E2E, CI/CD GitHub Actions) y guía de arranque local.
BILLING_REPORTS_MODULE.md	Especificación editorial del módulo de facturación: membrete institucional, bloques de la factura, y exportación en 4 formatos (PDF/A4, Word OpenXML, Excel, CSV UTF-8 BOM).
EHR_CLINICAL_MODULE.md	Flujo clínico completo (recepción → consulta EHR → estudios → vacunas → quirófano), visor de placas RX con inversión de contraste, carnet QR dinámico y guardia 24/7.
DATABASE_SCHEMA.md	Modelo entidad-relación en Mermaid y especificación de las 11 tablas; remite al diagrama editable <code>database-schema.excalidraw</code> .
RAILWAY_DEPLOYMENT_ENV.md	Guía de despliegue en Railway/Render con el listado completo de variables de entorno para el backend.

Verificación: documentación vs. implementación real

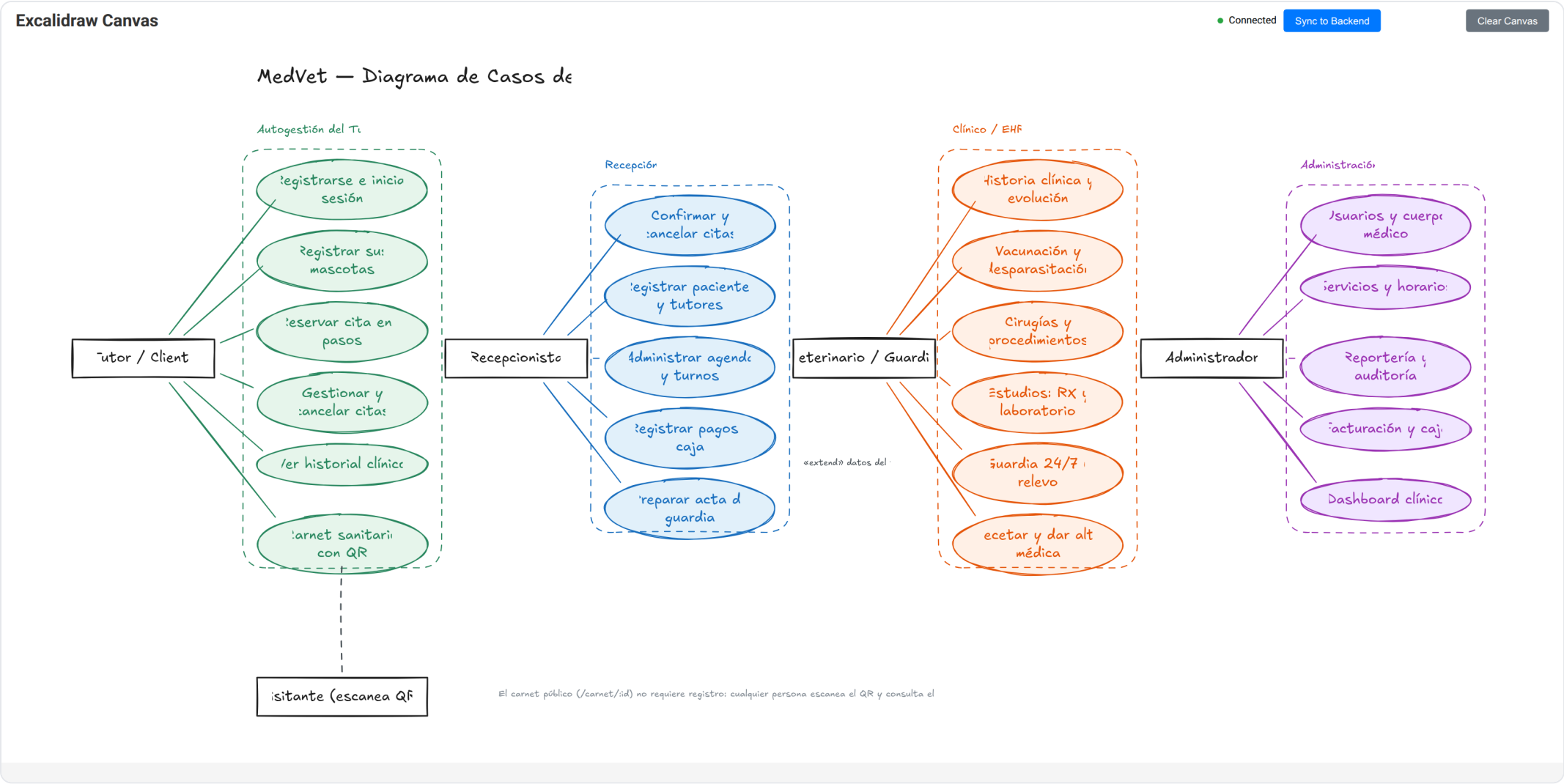
Lo que la documentación declara	Estado en el código	Veredicto
---------------------------------	---------------------	-----------

Motor de agenda available-slots , WebSocket, RBAC	Servicio existente y conectado; canales y hooks implementados	coherente
EHR, carnets, vacunación, guardia 24/7 (11 tablas)	Migraciones y seeds presentes; servicios operativos	coherente
«Fase 4: Facturación multimoneda — 100 % completada»	El frontend existe, pero no hay backend de pagos ni tabla de facturas ; los datos son maquetas	sobre-declarado
Exportación PDF/Word/Excel/CSV con membrete	Generación realizada en el navegador (frontend); sin backend de comprobantes fiscales	parcial
PostgreSQL 16, puertos 5432/5434/6379, Redis 6379	El despliegue real usa PostgreSQL 18 y Redis en puertos mapeados distintos (4533/6373)	detalle a unificar
Caché de slots en Redis con TTL 60 s	Redis está integrado, pero el caché de slots no es verificable en el código actual	confirmar

Conclusión: la documentación es excepcionalmente completa y coherente para las áreas implementadas (core clínico y de agenda). El único punto de divergencia relevante es el módulo de facturación/pagos, declarado como entregado cuando en realidad es una maqueta de frontend — que coincide exactamente con el ítem #1 de la deuda técnica y la prioridad de la hoja de ruta.

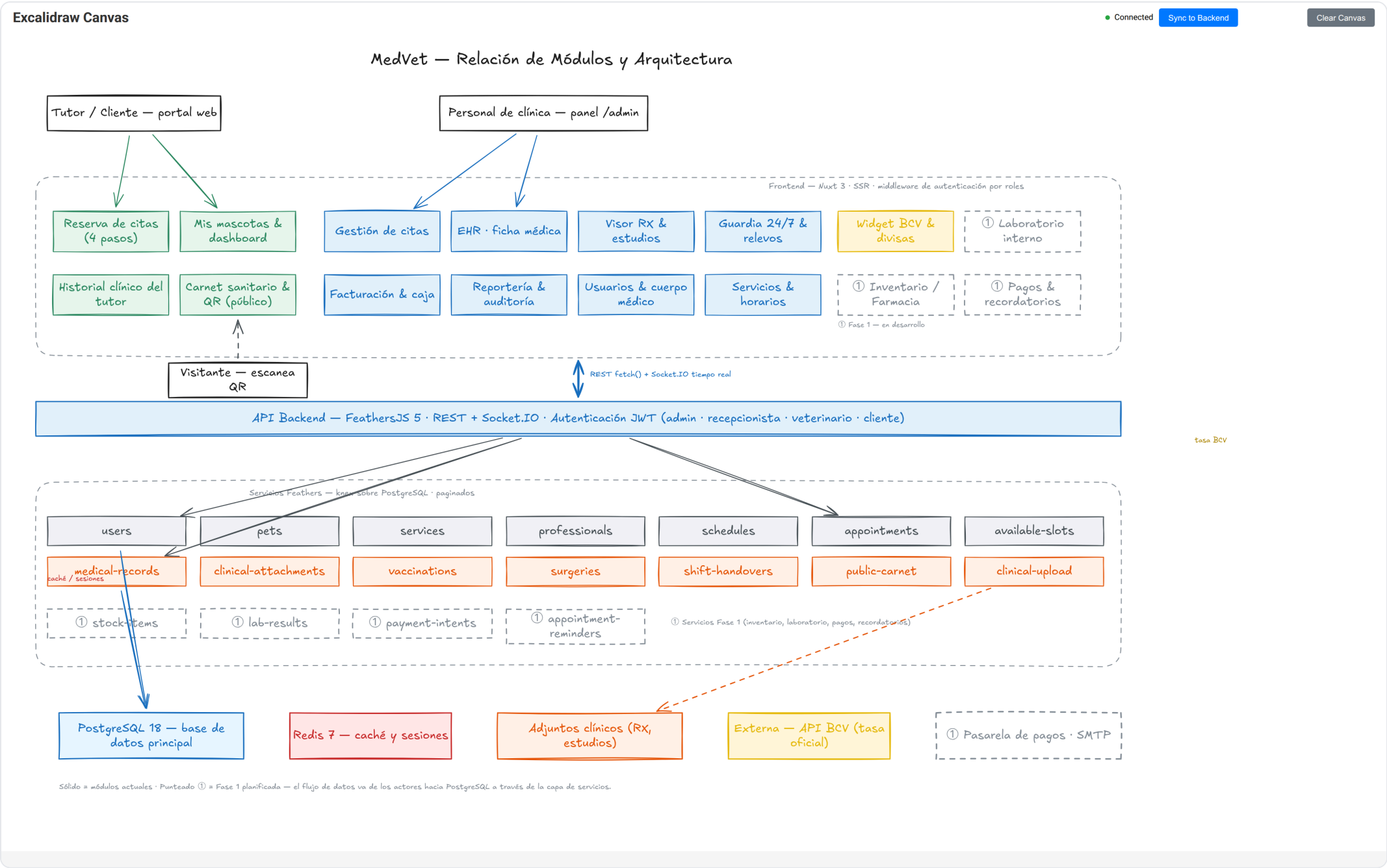
Anexo A — Diagrama de Casos de Uso

Actores (tutor, recepcionista, veterinario/guardia, administrador y visitante anónimo) y sus interacciones con el sistema.



Anexo B — Relación de Módulos y Arquitectura

Capas del sistema: actores → frontend Nuxt 3 → API FeathersJS → servicios → PostgreSQL/Redis. Módulos punteados (①): Fase 1 planificada.



Anexo C — Modelo Entidad-Relación

11 tablas actuales (PostgreSQL 18) agrupadas por dominio, con claves foráneas, más las 5 tablas propuestas de Fase 1 (punteadas).

